

Snowflake

Complete Data Platform Guide

For Data Engineers

1. WHAT IS SNOWFLAKE?

Snowflake is a cloud-native data platform (not just a data warehouse). It runs on AWS, Azure, and GCP infrastructure but provides a unified, cloud-agnostic SQL interface. **Key Philosophy:**

- **Separation of Storage and Compute:** Store once, compute independently
- **Multi-cluster shared data:** Multiple teams query same data concurrently without contention
- **Zero management:** No indexes, partitions, or vacuuming to manage
- **Pay for what you use:** Compute and storage billed separately

Snowflake vs Traditional DW:

- **Traditional (Oracle, Teradata):** Expensive, on-prem, tightly coupled storage/compute
- **Snowflake:** Cloud-native, auto-scaling, serverless feel with provisioned clusters

2. SNOWFLAKE ARCHITECTURE

Three-Layer Architecture:

Layer	Purpose	Details
1. Storage Layer	Centralized data storage	• Cloud object storage (S3/Blob/GCS) • Columnar format (proprietary) • Automatically compressed • Immutable (micro-partitions)
2. Compute Layer (Virtual Warehouses)	Query execution	• Independent compute clusters • Auto-suspend when idle • Auto-resume on query • Scale up/down/out
3. Cloud Services	Metadata, security, optimization	• Query parsing & optimization • Metadata management • Authentication (SSO, MFA) • Access control (RBAC)

Key Advantage:

Data stored once in central layer. Multiple virtual warehouses (ETL team, analytics team, ML team) can query simultaneously without impacting each other. Each warehouse has independent compute resources.

3. VIRTUAL WAREHOUSES (Compute)

What is a Virtual Warehouse?

A cluster of compute resources that executes queries. Think of it as a 'compute engine' that you can turn on/off and resize.

Size	Credits/Hour	Servers	Use Case
X-Small	1	1	Small queries, dev/test
Small	2	2	Light workloads
Medium	4	4	Standard workloads
Large	8	8	Heavy ETL, large scans
X-Large	16	16	Very heavy workloads
2X-Large	32	32	Massive transformations
3X-Large	64	64	Extreme workloads
4X-Large	128	128	Largest workloads

Key Features:

- **Auto-Suspend:** Warehouse stops after X minutes of inactivity (saves cost)
- **Auto-Resume:** Automatically starts when query submitted
- **Multi-Cluster:** Automatically adds clusters to handle concurrency
- **Scaling:** Change size any time (takes seconds)

Cost Example:

- Medium warehouse (4 credits/hour) running 8 hours/day = 32 credits/day
- If suspended 16 hours/day = \$0 cost for idle time
- Credit cost varies by region/edition (~\$2-4 per credit)

4. STORAGE & MICRO-PARTITIONS

Micro-Partitions:

Snowflake automatically organizes data into immutable micro-partitions (50-500 MB compressed). **How it Works:** • Data loaded → automatically partitioned by ingestion order (no explicit partitioning needed) • Each micro-partition stores metadata (min/max values, null count, distinct count) • **Pruning:** Query optimizer skips micro-partitions based on metadata (like partition elimination) **Clustering Keys (Optional):** • For very large tables (multi-TB), define clustering key to improve pruning • Snowflake auto-maintains clustering (re-clusters in background) • Example: CLUSTER BY (date, region) for sales table

Time Travel & Fail-Safe:

Time Travel (1-90 days): • Query historical data: `SELECT * FROM table AT(TIMESTAMP => '2024-01-01'::TIMESTAMP)` • Restore dropped table: `UNDROP TABLE my_table` • Clone at historical point: `CREATE TABLE new_table CLONE old_table AT(OFFSET => -3600)` **Fail-Safe (7 days after Time Travel):** • Additional 7-day protection (Snowflake can recover, you can't access directly) • Disaster recovery only (contact Snowflake support) **Cost Impact:** • Both Time Travel and Fail-Safe data stored = storage cost • Enterprise edition: Up to 90 days Time Travel (Standard: 1 day)

5. DATA LOADING METHODS

Method	Use Case	Details
COPY INTO	Bulk loading from cloud storage	• Load from S3/GCS/Azure Blob • CSV, JSON, Parquet, Avro, ORC • Can be automated with Snowpipe
Snowpipe	Continuous, automated loading	• Serverless ingestion • Triggered by file arrival (S3 event) • Near real-time (minutes)
INSERT/UPDATE	Small transactional changes	• Standard SQL DML • Not optimized for bulk (use COPY)
External Tables	Query without loading	• Like BigQuery external tables • Data stays in S3/GCS • Slower than native tables
Streams & Tasks	CDC / incremental processing	• Stream: Tracks changes (inserts/updates/deletes) • Task: Scheduled SQL execution

Example - COPY INTO:

```
COPY INTO my_table FROM @my_s3_stage/data/ FILE_FORMAT = (TYPE = 'PARQUET') PATTERN =  
'.*sales.*[.]parquet' ON_ERROR = 'CONTINUE';
```

6. STREAMS & TASKS (CDC Pipeline)

Streams:

A stream tracks DML changes (inserts, updates, deletes) on a table. Think of it as a change log. **Example:** CREATE STREAM my_stream ON TABLE source_table; -- Stream tracks all changes to source_table -- Query stream to see changes: SELECT * FROM my_stream; -- Columns: METADATA\$ACTION (INSERT/DELETE), METADATA\$ISUPDATE, METADATA\$ROW_ID -- Once consumed (inserted into target), stream offset advances

Tasks:

Scheduled SQL execution (like cron jobs). Can chain tasks into DAGs. **Example:** CREATE TASK my_task WAREHOUSE = ETL_WH SCHEDULE = '5 MINUTE' AS INSERT INTO target_table SELECT * FROM my_stream WHERE METADATA\$ACTION = 'INSERT'; -- Task runs every 5 minutes, processes changes from stream **Task Tree (Dependencies):** CREATE TASK child_task WAREHOUSE = ETL_WH AFTER parent_task -- Runs after parent_task completes AS MERGE INTO summary_table ...;

GCP Equivalent:

Streams + Tasks ≈ Cloud Composer (Airflow) for orchestration + incremental processing. Snowflake's approach is SQL-native and integrated, Airflow is more flexible for complex workflows.

7. ZERO-COPY CLONING

Create instant copy of table, schema, or database WITHOUT duplicating data. **How it Works:** • Metadata-only operation (pointers to micro-partitions) • Instant (even for multi-TB tables) • Storage cost ONLY for changes (copy-on-write) **Use Cases:** • **Dev/Test environments:** Clone production to staging instantly • **Experimentation:** Test transformations without affecting production • **Backups:** Point-in-time snapshot • **ML feature stores:** Clone for training data **Examples:** -- Clone table `CREATE TABLE dev_sales CLONE prod_sales;` -- Clone entire database `CREATE DATABASE dev_db CLONE prod_db;` -- Clone at specific time `CREATE TABLE backup CLONE prod_table AT(TIMESTAMP => '2024-01-01'::TIMESTAMP);`

Cost:

Initial clone = \$0 (metadata only). As clone diverges (updates/inserts), new micro-partitions created = storage cost. If clone never modified, storage cost = \$0.

8. DATA SHARING (Unique Feature)

Share live data with other Snowflake accounts WITHOUT copying data or ETL pipelines. **How it Works:** • Provider creates "Share" (grants read access to objects) • Consumer adds share as database (queries like local data) • **Zero data movement:** Consumer queries provider's storage layer • **Secure:** Provider controls access, can revoke any time **Use Cases:** • **B2B data products:** Sell/share data with customers • **Cross-department:** Marketing shares data with Sales (different accounts) • **Vendor collaboration:** Share with partners without ETL **Example (Provider):** CREATE SHARE sales_share; GRANT USAGE ON DATABASE sales_db TO SHARE sales_share; GRANT SELECT ON sales_db.public.transactions TO SHARE sales_share; ALTER SHARE sales_share ADD ACCOUNTS = consumer_account; **Example (Consumer):** CREATE DATABASE shared_sales FROM SHARE provider_account.sales_share; SELECT * FROM shared_sales.public.transactions; -- Query provider's data!

Comparison:

This is unique to Snowflake. GCP/AWS/Azure require data export/import or complex access delegation. Snowflake's architecture enables true live data sharing.

9. PERFORMANCE OPTIMIZATION

Technique	How	Impact
Cluster Keys	CLUSTER BY (date, region) for large tables (TB+)	Improves pruning, faster queries
Result Caching	Automatic (24 hours) Identical query = instant	Free, instant for repeated queries
Materialized Views	CREATE MATERIALIZED VIEW Pre-compute aggregates	Fast queries, auto-maintained
Search Optimization	ALTER TABLE ... ADD SEARCH OPTIMIZATION For point lookups	10-100x faster for selective queries
Warehouse Sizing	Start small, scale up if slow Monitor query time	Right-size compute cost
Multi-Cluster WH	Auto-scale for concurrency (Enterprise edition)	Handle spikes without queuing
Query Profiling	EXPLAIN PLAN QUERY_HISTORY view	Identify bottlenecks

Key Insight:

Snowflake is 'zero-tuning' for 80% of workloads (no indexes, partitions, vacuuming). For remaining 20% (multi-TB tables, high concurrency), use cluster keys, search optimization, and multi-cluster warehouses.

10. SNOWFLAKE vs BIGQUERY

Feature	Snowflake	BigQuery
Architecture	Separate storage/compute Provisioned warehouses	Serverless Auto-scaling
Pricing	Storage + compute (credits) Pay when warehouse running	Storage + query Pay per TB scanned
Compute Control	Explicit (start/stop WH)	Automatic (no control)
Concurrency	Multi-cluster warehouses (Enterprise)	Unlimited (automatic)
Time Travel	1-90 days (configurable)	7 days (default)
Cloning	Zero-copy cloning	Table snapshots (copy data)
Data Sharing	Native, live sharing	External tables or export
SQL Dialect	Standard SQL + extensions	Standard SQL
Materialized Views	Auto-maintained	Auto-maintained
ETL	Streams + Tasks (SQL-based)	Scheduled queries or Dataflow/Dataproc
Cost Predictability	High (know warehouse cost)	Lower (varies by query)
Multi-Cloud	AWS, Azure, GCP	GCP only

When to Choose:

Choose Snowflake if: • Need multi-cloud (run on AWS, Azure, GCP) • Want explicit compute control (cost predictability) • Need data sharing capabilities • Enterprise features (row-level security, multi-cluster) **Choose BigQuery if:** • Pure GCP environment • Want true serverless (zero management) • Prefer pay-per-query model • Already invested in GCP ecosystem

11. COST OPTIMIZATION

- 1. Auto-Suspend Warehouses:** Set to 1-5 minutes for interactive, 0 for ETL (manual control)
- 2. Right-Size Warehouses:** Start small (X-Small), scale up if slow. Bigger ≠ always faster
- 3. Separate Workloads:** Different warehouses for ETL, analytics, reporting (isolate resource contention)
- 4. Use Result Cache:** Identical queries are free (cached 24 hours)
- 5. Monitor Query History:** ACCOUNT_USAGE.QUERY_HISTORY - find expensive queries
- 6. Clustering:** Only for very large tables (TB+) with slow queries
- 7. Materialized Views:** Pre-compute common aggregations (cost = storage + refresh compute)
- 8. Resource Monitors:** Set credit limits to prevent runaway costs
- 9. Storage Lifecycle:** Drop old tables, reduce Time Travel retention (Standard: 1 day is often enough)
- 10. Enterprise Features:** Multi-cluster auto-scaling only if needed (adds cost but handles spikes)

Cost Monitoring Query:

```
-- Top 10 most expensive queries this month
SELECT query_text, warehouse_name, total_elapsed_time/1000 AS
seconds, credits_used_cloud_services FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY WHERE start_time >=
DATEADD(month, -1, CURRENT_TIMESTAMP()) ORDER BY credits_used_cloud_services DESC LIMIT 10;
```

12. SECURITY & GOVERNANCE

Feature	Description	Use Case
RBAC	Role-based access control Grant privileges to roles	Standard access management
Row-Level Security	Secure views with filtering Or Row Access Policies	Restrict data by user/role
Column Masking	Dynamic Data Masking Or masking policies	Hide PII (SSN, email)
Multi-Factor Auth	MFA for login	Enhanced security
SSO	SAML 2.0 integration (Okta, Azure AD, etc.)	Enterprise authentication
Network Policies	IP whitelist/blacklist	Restrict access by IP
Data Encryption	End-to-end encryption (at rest + in transit)	Always encrypted (automatic)
Audit Logging	ACCOUNT_USAGE views (login history, query history)	Compliance auditing

Row-Level Security Example:

```
-- Create secure view CREATE SECURE VIEW sales_filtered AS SELECT * FROM sales WHERE region = CURRENT_ROLE();
-- User sees only their region -- Or use Row Access Policy (Enterprise) CREATE ROW ACCESS POLICY region_policy
AS (region VARCHAR) RETURNS BOOLEAN -> CASE WHEN CURRENT_ROLE() = 'ADMIN' THEN TRUE WHEN region =
CURRENT_USER() THEN TRUE ELSE FALSE END; ALTER TABLE sales ADD ROW ACCESS POLICY region_policy ON (region);
```

13. INTERVIEW QUESTIONS & ANSWERS

Question	Answer
What is Snowflake's architecture?	Three-layer architecture: (1) Storage layer (cloud object storage with micro-partitions), (2) Compute layer (virtual warehouses), (3) Cloud services layer (metadata, authentication, etc.).
How does Snowflake handle concurrency?	Multi-cluster warehouses (Enterprise edition) automatically add clusters when queries queue. Each cluster is independent and can run queries in parallel.
What are micro-partitions?	Snowflake auto-organizes data into immutable 50-500MB compressed blocks. Each stores metadata (min/max, etc.) for efficient pruning.
Zero-copy cloning?	Metadata-only operation creating instant copy without duplicating data. Uses copy-on-write for new/modified data.
Time Travel use cases?	Query historical data (audit), restore dropped tables (UNDROP), clone at specific point in time. Configurable 1-90 days retention.
Streams vs CDC?	Stream tracks DML changes (insert/update/delete) on table. Pair with Tasks for scheduled processing. Equivalent to CDC.
How to optimize slow queries?	Check EXPLAIN plan for partition pruning. Add cluster key for large tables (TB+). Use search optimization for point queries.
Snowflake vs BigQuery?	Snowflake: provisioned compute (explicit control, cost predictable), multi-cloud. BigQuery: serverless (no management), Google Cloud only.

14. GCP → SNOWFLAKE TRANSLATION

GCP Concept	Snowflake Equivalent	Notes
BigQuery Dataset	Snowflake Schema	Organizational container for tables
BigQuery Table	Snowflake Table	Same concept, different storage
Partitioned Table	Clustered Table	BigQuery: explicit partitioning Snowflake: micro-partitions (auto)
Clustered Table	Clustered Table	Similar concept (colocate data)
Scheduled Queries	Tasks	Snowflake: more powerful (DAGs)
Streaming Inserts	Snowpipe	Snowflake: file-based (S3 events)
DML (INSERT/UPDATE)	DML	Same SQL syntax
External Tables	External Tables	Query GCS without loading
Table Snapshots	Zero-Copy Clone	Snowflake: instant, no storage cost
Time Travel (7 days)	Time Travel (1-90 days)	Snowflake: configurable retention

Migration Path:

1. **Export from BigQuery:** EXPORT DATA to GCS (Parquet format)
2. **Create Snowflake Stage:** CREATE STAGE pointing to GCS bucket
3. **COPY INTO Snowflake:** COPY INTO table FROM @stage
4. **Recreate Views/Materialized Views:** SQL mostly compatible
5. **Migrate Scheduled Queries → Tasks:** Rewrite as Tasks + Streams for incremental
6. **Update BI Tools:** Point Looker/Tableau to Snowflake (JDBC/ODBC)

15. KEY TAKEAWAYS

- 1. Architecture:** Storage/compute separation = concurrent workloads without interference
- 2. Virtual Warehouses:** Independent compute clusters, auto-suspend/resume, scale up/down
- 3. Zero Management:** No indexes, partitions, or vacuuming (micro-partitions automatic)
- 4. Zero-Copy Cloning:** Instant copies for dev/test/backup (unique to Snowflake)
- 5. Time Travel:** Query historical data, restore dropped objects (1-90 days)
- 6. Data Sharing:** Live sharing without data movement (unique capability)
- 7. Streams + Tasks:** CDC and orchestration in SQL (alternative to Airflow for simple cases)
- 8. Multi-Cloud:** Runs on AWS, Azure, GCP (BigQuery is GCP-only)
- 9. Cost Model:** Storage + compute credits (predictable vs BigQuery pay-per-query)
- 10. Enterprise Ready:** RBAC, row-level security, masking, audit logging

For BigQuery Users:

Snowflake gives you more control (warehouses, cost) but requires more management. BigQuery is truly serverless. Snowflake's unique advantages: multi-cloud, data sharing, zero-copy cloning. Both are excellent data warehouses - choice depends on cloud strategy and control preferences.